

JavaScript - Day 8

Networking | Asynchronous Programming | Promises

Fetch API | Async / Await

Student Notes — Real-World Projects & Interview Reference

Covers how browsers talk to servers, how JavaScript handles background tasks, how to work with Promises, and how to write clean async code with Fetch and Async/Await.

Day 8 Learning Objectives

By the end of this session, students should be able to:

- ✓ Understand how browsers communicate with servers
- ✓ Understand synchronous vs asynchronous programming
- ✓ Work with JavaScript Promises
- ✓ Fetch data from APIs using Fetch API
- ✓ Write modern asynchronous code using Async/Await
- ✓ Handle API errors properly
- ✓ Understand how modern web applications retrieve and display data

1. Networking

How browsers communicate with servers

What is Networking?

Networking is the communication between a client (frontend browser) and a server (backend). Every time you open a website, watch a video, or shop online — networking is happening.

```
Client (Browser)
  ↓
Request sent to Server
  ↓
Server processes request
  ↓
Response sent back
  ↓
Client displays the data
```

Real-world examples:

- Opening Google
- Watching YouTube
- Using Instagram
- Shopping on Amazon

HTTP (HyperText Transfer Protocol)

HTTP is the protocol (set of rules) used for communication between client and server.

Request

Sent from client to server. Contains:

- URL — where to send the request
- Method — GET, POST, PUT, DELETE
- Headers — metadata about the request
- Body (optional) — data to send

Response

Sent from server back to the client. Contains:

- Status Code — 200, 404, 500, etc.
- Headers — metadata about the response
- Data — the actual content returned

URL Structure

```
https://jsonplaceholder.typicode.com/todos/1
```

Part	Value	Meaning
Protocol	https	Secure HTTP connection
Domain	jsonplaceholder.typicode.com	The server address
Endpoint	/todos	The resource collection
Resource ID	/1	A specific item within the collection

HTTP Methods (CRUD)

Method	Operation	Real-World Example
GET	Read / Retrieve data	Get products, get users, get weather
POST	Create new data	Create user, add product, create post
PUT	Update existing data	Update profile, update product
DELETE	Delete data	Delete user, delete product

HTTP Status Codes

Status codes tell the client what happened with the request.

1xx — Informational

100	Continue
101	Switching Protocols

2xx — Success

200	OK — request succeeded
201	Created — new resource created
204	No Content — success with no data

3xx — Redirection

301	Moved Permanently
304	Not Modified

4xx — Client Error

400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found

5xx — Server Error

500	Internal Server Error
503	Service Unavailable
504	Gateway Timeout

Web APIs

Web APIs are browser-provided features that extend JavaScript capabilities beyond what the language itself offers.

Web API	Usage
DOM API	Select and manipulate HTML elements
Fetch API	Make HTTP requests to servers
setTimeout()	Execute code after a delay
setInterval()	Execute code repeatedly at intervals
LocalStorage	Store data in the browser persistently

2. Asynchronous Programming

Running background tasks without blocking the user

What is Asynchronous Programming?

Asynchronous Programming allows JavaScript to perform time-consuming tasks in the background without freezing or blocking other code. The rest of the program continues running.

Real-Life Analogy

```

Order food at a restaurant
    ↓
Continue talking with friends
    ↓
Food arrives later – you eat it then
  
```

You don't stare at the kitchen wall waiting. You continue while the food is being prepared.

Why Do We Need It?

Some operations take time:

- API Calls — waiting for server response
- Database Queries
- File Uploads
- Timers and delays

Without Async	With Async
Application freezes	User can keep interacting

Without Async	With Async
Other code stops	Other code continues
Poor user experience	Smooth user experience

Synchronous vs Asynchronous — Code Comparison

Synchronous Execution

Code executes strictly line by line. Each line waits for the previous one to complete.

```
console.log("Step 1");  
console.log("Step 2");  
console.log("Step 3");
```

Output:

```
Step 1  
Step 2  
Step 3
```

Asynchronous Execution

Time-consuming tasks are moved to the background. JavaScript continues to the next line immediately.

```
console.log("Step 1");  
  
setTimeout(() => {  
  console.log("Step 2");  
}, 3000);  
  
console.log("Step 3");
```

Output:

```
Step 1  
Step 3  
Step 2 (appears 3 seconds later)
```

Timers

setTimeout()

Executes code once after a specified delay.

```
setTimeout(() => {
  console.log("Hello");
}, 2000) // runs after 2 seconds
```

setInterval()

Runs code repeatedly at a specified interval.

```
setInterval(() => {
  console.log("Running...");
}, 1000) // runs every 1 second
```

clearInterval()

Stops a running interval.

```
clearInterval(intervalId);
```

JavaScript is Single-Threaded

JavaScript executes one statement at a time using a single Call Stack. It cannot do two things at exactly the same moment — but it can schedule tasks to run later using the Event Loop.

How Asynchronous Programming Works Internally

```
JavaScript Call Stack
↓
Async task handed to Web APIs
↓
Web APIs handle the task in the background
↓
Result placed in Callback Queue
↓
Event Loop moves it to Call Stack when free
↓
Code executes
```

Ways to Achieve Asynchronous Programming

Method	Description
Callbacks	Oldest method — functions passed as arguments
Promises	Cleaner — handles future values with <code>.then()/.catch()</code>
Async/Await	Cleanest — reads like synchronous code

Method	Description
Timers	setTimeout / setInterval
Event Listeners	addEventListener() callbacks

3. Promises

Handling future values in asynchronous operations

What is a Promise?

A Promise is a JavaScript object that represents the eventual success or failure of an asynchronous operation. It holds a future value that is not yet available.

Real-Life Analogy

Friend says: "I promise I'll send the notes tonight"

Two possible outcomes:

Notes Sent → Promise Fulfilled (Success)

Notes Not Sent → Promise Rejected (Failure)

Why Promises?

- Cleaner code compared to nested callbacks
- Better error handling
- Avoids Callback Hell
- Chainable for sequential operations

Promise Syntax

```
const promise = new Promise((resolve, reject) => {  
  
  if(success) {  
    resolve("Operation succeeded");  
  }  
  else {  
    reject("Operation failed");  
  }  
  
});
```


Promise States

State	Meaning	Triggered By
Pending	Waiting — operation not yet complete	Initial state
Fulfilled	Operation completed successfully	resolve() called
Rejected	Operation failed	reject() called

```

graph TD
    Pending -->|resolve| Fulfilled
    Pending -->|reject| Rejected
  
```

Producing vs Consuming a Promise

Role	Code	Responsibility
Producing	<code>const p = new Promise(...)</code>	Creates and resolves/rejects the promise
Consuming	<code>p.then(...).catch(...)</code>	Uses the result when ready

then() — Handles Success

```
promise.then(data => {
  console.log(data);
});
```

catch() — Handles Failure

```
promise.catch(error => {
  console.log(error);
});
```

Promise Chaining

Promise chaining allows multiple asynchronous operations to run one after another in a readable sequence.

```
promise
  .then(result => result * 2)    // 10 → 20
  .then(result => result + 5)    // 20 → 25
  .then(result => console.log(result));
```

Output:

25

Callback Hell — The Problem Promises Solve

✗ Wrong approach — deeply nested callbacks

```
task1(() => {
  task2(() => {
    task3(() => {
      task4(() => {
        // Impossible to read or debug
      });
    });
  });
});
```

✓ Better approach — Promise chaining

```
task1()
  .then(() => task2())
  .then(() => task3())
  .then(() => task4())
  .catch(err => console.log(err));
```

4. Fetch API

Making HTTP requests from JavaScript

What is Fetch API?

Fetch API is a built-in browser Web API used to make HTTP requests to servers. It is used in Login Systems, Weather Apps, E-Commerce, Social Media Apps — anywhere data comes from a server.

Two Key Facts About Fetch

- ✓ Fetch API is a Web API — provided by the browser, not JavaScript itself
- ✓ Fetch API always returns a Promise

Basic Fetch Syntax

```
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log(error));
```

Fetch Flow

```
fetch(url) called
  ↓
HTTP Request sent to Server
  ↓
Server responds with HTTP Response
  ↓
response.json() converts body to JS Object
  ↓
Data is ready to use
```

response.json()

Note: *response.json() also returns a Promise. That is why we need a second .then() to access the actual data.*

GET Request

Used to retrieve data from a server.

```
const url = "https://jsonplaceholder.typicode.com/todos/1";

fetch(url)
  .then(res => res.json())
  .then(data => {
    console.log(data.title);
    console.log(data.completed);
  })
  .catch(error => console.log("Error:", error));
```

POST Request

Used to send new data to a server.

```
fetch(url, {
  method: "POST",
  body: JSON.stringify({
    name: "John",
    email: "john@gmail.com"
  }),
  headers: {
```

```

    "Content-Type": "application/json"
  }
})
  .then(res => res.json())
  .then(data => console.log(data));

```

Request Headers

Headers provide metadata about the request.

Header	Purpose
Content-Type: application/json	Tells server the body is JSON
Authorization: Bearer Token	Sends authentication token
Accept: application/json	Tells server what format to return

Error Handling in Fetch

Common Misconception

⚠ Warning: `fetch().catch()` does NOT catch HTTP errors like 404 or 500. The `catch()` block only fires for network failures (e.g. no internet). A 404 response is still a valid HTTP response — `fetch()` considers it a success!

```

// 404 does NOT go into catch()
fetch("https://api.example.com/missing")
  .then(res => console.log(res.status)) // prints 404
  .catch(err => console.log(err));      // never reached

```

The Fix — `response.ok`

`response.ok` is true when the status code is 200–299. It is false for any other status code including 404, 500, 403 etc.

```

fetch(url)
  .then(res => {
    if(!res.ok) {
      throw new Error("HTTP Error: " + res.status);
    }
    return res.json();
  })
  .then(data => console.log(data))
  .catch(err => console.log(err.message));

```

response.ok Value	Status Code Range	Meaning
true	200 – 299	Request succeeded

false	400, 401, 403, 404, 500...	Error occurred — handle it
-------	----------------------------	----------------------------

5. Async / Await

Writing cleaner Promise-based code

What is Async/Await?

Async/Await is modern JavaScript syntax that makes asynchronous code look and behave like synchronous code. It is built entirely on top of Promises — not a replacement for them.

Why Async/Await?

Promise Style	Async/Await Style
.then() → chain	await → same result, no chaining
.catch() → error handling	try/catch → familiar error handling
Harder to read deep chains	Reads top-to-bottom like normal code

Side-by-Side Comparison

Using Promises

```
fetch(url)
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.log(err));
```

Using Async/Await

```
async function getData() {
  try {
    const res = await fetch(url);
    const data = await res.json();
    console.log(data);
  }
  catch(err) {
    console.log(err);
  }
}
```

The async Keyword

The `async` keyword makes a function asynchronous. Every `async` function automatically returns a `Promise`, even if you don't explicitly return one.

```
async function getData() {
  // code here
}

// Also works with arrow functions:
const getData = async () => {
  // code here
}
```

Note: Every `async` function always returns a `Promise` — even if it returns a plain value.

The `await` Keyword

`await` pauses the execution of an `async` function until a `Promise` resolves. It then returns the resolved value. `await` can only be used inside an `async` function.

```
const result = await somePromise;
// Pauses here until promise resolves
// result = the resolved value
```

⚠ Warning: `await` can ONLY be used inside `async` functions. Using `await` outside an `async` function will cause a `SyntaxError`.

Converting `.then()/catch()` to `Async/Await`

Step	Before (<code>.then</code>)	After (<code>await</code>)
1	<code>function getData() {...}</code>	<code>async function getData() {...}</code>
2	<code>fetch(url).then(res => ...)</code>	<code>const res = await fetch(url)</code>
3	<code>res.json().then(data => ...)</code>	<code>const data = await res.json()</code>
4	<code>.catch(err => ...)</code>	<code>try { } catch(err) { }</code>

Why Are There Two `await` Statements?

```
const res = await fetch(url);    // First await
```

```
const data = await res.json();    // Second await
```

await	Waits For	Returns
1st — await fetch(url)	The HTTP response to arrive from server	Response object
2nd — await res.json()	JSON body to be parsed into JS object	JavaScript object

Complete Async/Await with Error Handling

```
async function getUser() {
  try {
    const res = await fetch("https://api.github.com/users/octocat");

    if(!res.ok) {
      throw new Error("HTTP Error: " + res.status);
    }

    const data = await res.json();

    console.log(data.name);
    console.log(data.followers);
  }
  catch(err) {
    console.log("Something went wrong:", err.message);
  }
}

getUser();
```

Async/Await Internal Working

```
Async/Await syntax
  ↓
Compiled to Promises internally
  ↓
Promises use the Event Loop
  ↓
Event Loop uses Web APIs
  ↓
Web APIs handle the background work
```

Note: *Async/Await is not a replacement for Promises. It is syntactic sugar — a cleaner way to write the same Promise-based code.*

Complete Day 8 — Concept Flow

```
Networking – Client needs data from a server
  ↓
HTTP – Rules for sending request and receiving response
  ↓
Asynchronous Programming – Don't freeze while waiting
  ↓
Promises – Handle future results cleanly
  ↓
Fetch API – Make the actual HTTP request
  ↓
response.ok – Properly detect HTTP errors
  ↓
Async/Await – Write cleaner, readable async code
```

Common Interview Questions

Networking

1. What is HTTP?
2. Difference between Request and Response?
3. Explain GET, POST, PUT, and DELETE methods.
4. What are HTTP status codes? Give examples.
5. What are Web APIs?

Asynchronous Programming

6. What is asynchronous programming?
7. Difference between synchronous and asynchronous programming?
8. Is JavaScript single-threaded? How does async work then?
9. What is `setTimeout()`?
10. What is `setInterval()`? How is it stopped?
11. Explain the Event Loop.

Promises

12. What is a Promise?

13. What are the three states of a Promise?
14. Difference between `resolve()` and `reject()`?
15. What is Promise chaining?
16. What is Callback Hell?
17. How do Promises solve Callback Hell?

Fetch API

18. What is Fetch API?
19. What does `fetch()` return?
20. What does `response.json()` return?
21. Why doesn't fetch treat 404 as an error?
22. What is `response.ok`?
23. How do you make a POST request using fetch?

Async/Await

24. What is Async/Await?
25. What does the `async` keyword do?
26. What does the `await` keyword do?
27. Can `await` be used outside `async` functions?
28. Does `async` always return a Promise?
29. Is Async/Await a replacement for Promises?
30. Why are there two `await` statements when using fetch?

Mini Assignment — GitHub User Search App

Build a GitHub User Search Application that fetches live data from the GitHub API.

Feature	Concept Used
Input field for GitHub username	HTML + Events
Search button triggers fetch	<code>addEventListener()</code>
Fetch data from GitHub API	Fetch API
Use <code>async/await</code> for cleaner code	Async/Await
Check <code>response.ok</code> before reading data	Error Handling
Display name, followers, following, repos, avatar	DOM Manipulation

Feature	Concept Used
Show friendly error if user not found	try/catch

API Endpoint to Use

```
https://api.github.com/users/{username}
```

Key Takeaways

- ✓ HTTP is the protocol for client-server communication.
- ✓ JavaScript is single-threaded but supports asynchronous behavior via the Event Loop.
- ✓ Promises handle future values — states are Pending, Fulfilled, and Rejected.
- ✓ Promise chaining allows sequential async operations without nesting.
- ✓ Fetch API is a Web API that returns a Promise.
- ✓ `response.json()` also returns a Promise — always use a second `await` or `.then()`.
- ✓ `fetch().catch()` only catches network failures — NOT 404 or 500 errors.
- ✓ Always use `response.ok` to properly detect HTTP errors.
- ✓ Async/Await is syntactic sugar built on top of Promises — not a replacement.
- ✓ Every async function automatically returns a Promise.
- ✓ `await` can only be used inside async functions.
- ✓ Two `await` statements are needed with `fetch` — one for the response, one for the JSON body.